Del incidente a la recuperación

Custodia de claves, cifrado híbrido y autenticación

Modelo A: el servidor genera y entrega la semilla maestra

Curso: Criptografía | **Actividad III**

Autores

César Roa

Sebastián Ibáñez

Construimos, en un laboratorio propio y aislado, el ciclo completo de un incidente de ransomware*: infección, intercambio de la semilla maestra, cifrado de los archivos, almacenamiento y recuperación. Sobre ese ciclo montamos las dos vías de recuperación (pago e investigación), el intermediario MITM y la defensa con firmas digitales.*

8 de noviembre de 2026

1 Introducción

Para esta actividad elegimos el **Modelo A**: es el servidor quien genera la semilla maestra s_V de cada víctima y se la entrega ya cifrada. Nos pareció el más natural para pensar la idea de *custodia de claves*, porque deja muy claro quién guarda el secreto (el servidor, como custodio) y quién lo usa sin conservarlo (la víctima, que cifra sus archivos y luego olvida la semilla).

La idea completa es la siguiente. La víctima se registra ante el servidor y recibe s_V protegida con RSA. Con esa semilla deriva una clave distinta para cada archivo y los cifra con AES. Después cierra todo y olvida la semilla; para volver a abrir sus archivos debe *recuperarla*, y hay dos caminos: pedirla al servidor mediante un evento de pago, o reconstruirla por su cuenta investigando el generador. A partir de ahí añadimos un atacante que se mete en medio de la comunicación (MITM) y, finalmente, la defensa que lo neutraliza usando firmas digitales.

A lo largo del informe explicamos qué tomamos del cuaderno de referencia (el Colab 1) y, sobre todo, **qué tuvimos que cambiar y por qué**, mostrando los fragmentos de código originales junto a los nuestros.

2 Cómo funciona nuestra aplicación

El corazón del sistema es un módulo común, `crypto_utils.py`, que reúne todas las operaciones criptográficas, y dos programas que hablan por *sockets*: `servidor.py` y `cliente.py`. El flujo sigue los seis pasos del enunciado.

Primero, cliente y servidor preparan su par de claves RSA-2048 y la víctima se registra enviando su identificador id_V y su clave pública pk_V; el servidor responde con la suya. Después, el servidor genera la semilla con el generador suministrado, la guarda asociada a esa víctima y se la manda cifrada con RSA-OAEP. La víctima la descifra con su clave privada y la mantiene únicamente en memoria. Con s_V ya disponible, para cada archivo $i = 0, 1, 2, \ldots$ deriva una clave

$$K_i = \text{HMAC-SHA256}(s_V,\ i),$$

cifra el archivo con AES-256-GCM (nonce aleatorio de 12 bytes y *tag* de 16) y guarda todo en un pequeño archivo `.enc.json`. Un detalle importante es que ligamos cada cifrado a su contexto mediante los datos asociados $\text{AAD} = id_V \parallel i$: así un archivo cifrado no puede "colarse" con otro índice o de otra víctima sin que la verificación falle. Terminado el cifrado, la víctima borra de memoria la semilla y las claves derivadas.

Para la recuperación por pago, la víctima vuelve a arrancar, carga de disco su identidad y sus archivos cifrados, y pide la semilla con un evento de autorización; el servidor, usando la clave pública que guardó en el registro, le devuelve s_V otra vez. Con ella regenera cada K_i, verifica el *tag* de GCM, guarda los archivos recuperados y los compara byte a byte contra los originales. Decidimos que la entrega inicial ocurra **una sola vez**: si la víctima intentara pedir de nuevo la semilla como "primera entrega", el servidor la rechaza y la obliga a pasar por la vía de pago, que es justamente lo que el enunciado busca proteger.

3 De dónde partimos: el Colab y cómo lo adaptamos

Nos apoyamos en el *Colab 1*, sobre todo en la sección 7 (RSA-OAEP) y la sección 8 (la aplicación cliente-servidor). De ahí tomamos la estructura del cifrado híbrido y la forma de comunicar los procesos. Ahora bien, el Colab usa la biblioteca `PyCryptodome` y resuelve un problema algo distinto (establecer una *clave de sesión*), mientras que el enunciado nos pide usar `cryptography` y trabajar con una *semilla maestra* de la que se derivan claves por archivo. Por eso, más que copiar, tuvimos que reescribir cada pieza. A continuación mostramos los fragmentos originales y los nuestros.

3.1 Transportar el secreto con RSA-OAEP

En el Colab, el secreto aleatorio se encapsula y se recupera con OAEP así:

Colab 1 (sección 8.2, PyCryptodome)

```
def encapsular_oaep(pk):
    secreto = secrets.token_bytes(32)
    publica = RSA.construct((pk["n"], pk["e"]))
    oaep = PKCS1_OAEP.new(publica, hashAlgo=SHA256)
    return oaep.encrypt(secreto), secreto

def desencapsular_oaep(sk, encapsulacion):
    privada = RSA.construct((sk["n"], sk["e"], sk["d"], sk["p"], sk["q"]))
    oaep = PKCS1_OAEP.new(privada, hashAlgo=SHA256)
    return oaep.decrypt(encapsulacion)
```

En nuestro proyecto hacemos lo mismo conceptualmente (cifrar y descifrar con OAEP y SHA-256), pero con `cryptography`, que trabaja con objetos de clave en lugar de tuplas de enteros:

Nuestro código (crypto_utils.py, cryptography)

```
_OAEP = padding.OAEP(mgf=padding.MGF1(hashes.SHA256()),
                     algorithm=hashes.SHA256(), label=None)

def rsa_cifrar(clave_publica, mensaje):
    return clave_publica.encrypt(mensaje, _OAEP)

def rsa_descifrar(clave_privada, cifrado):
    return clave_privada.decrypt(cifrado, _OAEP)
```

La operación criptográfica es la misma (OAEP con MGF1 y SHA-256). Lo que cambia es la biblioteca y, con ella, el manejo de las claves: el Colab reconstruye la clave desde sus componentes (`RSA.construct`), mientras que nosotros las serializamos en formato PEM y las cargamos como objetos. Además, en el Colab el secreto encapsulado es una clave de sesión que se genera ahí mismo; en el nuestro, el "secreto" que viaja dentro de OAEP es la semilla maestra s_V que produce el generador suministrado.

3.2 Derivar las claves: de HKDF a una clave por archivo

Esta es la diferencia de fondo. El Colab deriva *una* clave de sesión con HKDF a partir del secreto y una sal:

Colab 1 (sección 8.2, PyCryptodome)

```
def derivar_clave_sesion(secreto, sal):
    if len(sal) != 16:
        raise ValueError("La sal de sesion debe tener 16 bytes.")
    return HKDF(secreto, 32, sal, SHA256, context=b"Colab01/sesion")
```

Nosotros no queremos una única clave de sesión, sino una clave *distinta por archivo*, y además el enunciado fija exactamente la fórmula. Por eso usamos HMAC con el contador del archivo como mensaje:

Nuestro código (crypto_utils.py)

```
def derivar_clave(s_v, i):
    h = hmac.HMAC(s_v, hashes.SHA256())
    h.update(i.to_bytes(8, "big")) # K_i = HMAC-SHA256(s_v, i)
    return h.finalize()
```

Nuestra razón para cambiarlo es sencilla. Ambos esquemas derivan claves a partir de un secreto usando HMAC por debajo, pero con propósitos distintos. HKDF produce una sola clave

de sesión para toda la conexión; nosotros necesitamos que cada archivo i tenga su propia clave K_i, reproducible más tarde solo con la semilla y el índice. Así, en la recuperación basta volver a calcular $K_i = \mathrm{HMAC}(s_V, i)$ para cada archivo, sin guardar ninguna clave en disco. Esta decisión es también la que hace posible el ataque de investigación (5b): como K_0 depende solo de la semilla y de $i = 0$, quien adivine la semilla reconstruye la clave.

3.3 Cifrar con AES-GCM

El Colab cifra cada envío con AES-GCM y, para no repetir nonces, usa un contador como nonce:

Colab 1 (sección 8.3, PyCryptodome)

```
def cifrar_simetrico(clave, datos, contador):
    nonce = contador.to_bytes(12, "big")
    aes = AES.new(clave, AES.MODE_GCM, nonce=nonce, mac_len=16)
    cifrado, etiqueta = aes.encrypt_and_digest(datos)
    return nonce + etiqueta + cifrado
```

Nosotros también usamos AES-GCM con *tag* de 16 bytes, pero tomamos el nonce al azar y añadimos datos asociados que ligan el cifrado a la víctima y al índice:

Nuestro código (crypto_utils.py)

```
def construir_aad(id_v, i):
    return id_v.encode("utf-8") + b"|" + i.to_bytes(8, "big")

def aes_cifrar(clave, plaintext, aad):
    nonce = os.urandom(12)
    ct = AESGCM(clave).encrypt(nonce, plaintext, aad)
    return nonce, ct[:-16], ct[-16:]  # nonce, ciphertext, tag
```

La diferencia aquí está en el nonce. El Colab lo deriva del contador porque en una sesión corta eso basta para que no se repita y, de paso, le sirve para detectar paquetes fuera de orden. En nuestro caso cada archivo se cifra con una clave K_i distinta y queremos guardarlos de forma independiente, así que un nonce aleatorio es más simple y seguro (lo guardamos junto al cifrado). Donde el Colab usa el contador como control anti-repetición, nosotros usamos los datos asociados $id_V \parallel i$: no cifran nada, pero quedan "firmados" por el *tag*, de modo que si alguien cambia el identificador o el índice, el descifrado falla.

3.4 La comunicación cliente-servidor

De la sección 8 también tomamos la forma de enviar mensajes por TCP. Como TCP es un flujo de bytes sin fronteras, el Colab antepone a cada paquete su longitud en cuatro bytes. Nosotros hacemos exactamente eso, pero enviando objetos JSON (lo que nos da mensajes con campos claros como `accion` o `id_v`):

Nuestro código (crypto_utils.py)

```
def enviar_json(sock, obj):
    datos = json.dumps(obj).encode("utf-8")
    sock.sendall(struct.pack(">I", len(datos)) + datos) # longitud(4B) || datos
```

En resumen, del Colab conservamos el cifrado híbrido (OAEP para el secreto y AES-GCM para los datos) y el estilo de comunicación; cambiamos la biblioteca, la naturaleza del secreto (una semilla maestra en vez de una clave de sesión), la derivación (una clave por archivo en vez de una de sesión) y el manejo del nonce; y añadimos toda la parte de persistencia y de autenticación, que el Colab da por supuesta.

4 El protocolo con autenticación y el punto de rechazo

El enunciado nos pide adaptar el diagrama del modelo añadiendo los datos firmados, las verificaciones y el punto exacto donde se detiene un ataque. La Figura 1 muestra el intercambio legítimo ya autenticado y la Figura 2, el ataque del intermediario detenido por la defensa. Las claves públicas de verificación están **pre-compartidas** en `trusted_keys/` y no viajan por la red.

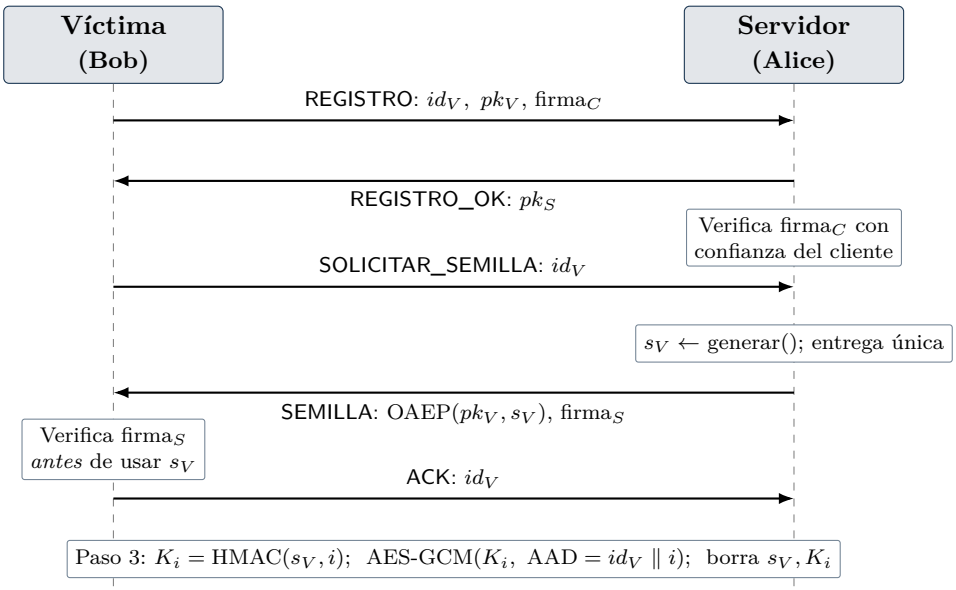


Figura 1: Intercambio legítimo autenticado. La recuperación por pago repite el patrón: el cliente firma `RECUPERAR` y el servidor devuelve OAEP(pk_V, s_V) firmado, que el cliente verifica antes de descifrar.

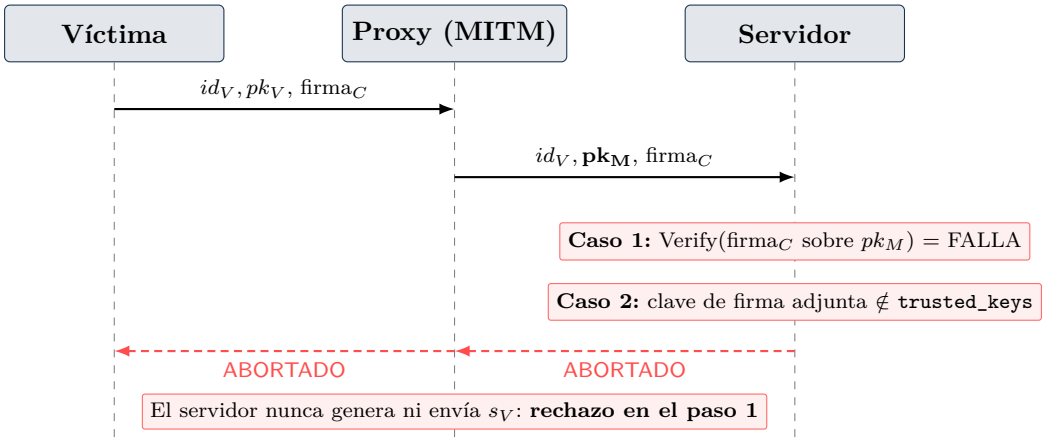


Figura 2: El ataque MITM detenido por la defensa. El servidor aborta antes de liberar la semilla porque valida exclusivamente contra `trusted_keys/`.

El rechazo ocurre en el **paso 1**, en el servidor, antes de generar o enviar la semilla. Lo explicamos en detalle en la sección de la defensa.

5 Rompiendo la semilla: análisis de entropía

La parte más interesante del trabajo es que, aun usando RSA-OAEP y AES-256-GCM (primitivas fuertes), el sistema se puede romper por completo. El motivo no está en el cifrado,

sino en el generador suministrado:

```
configuracion = secrets.randbelow(9) + 16 # c uniforme en {16,...,24}
valor         = secrets.randbelow(1 << configuracion) # uniforme en [0, 2^c)
return valor.to_bytes(32, "big")
```

Aunque la semilla ocupa 32 bytes, su valor nunca supera 2^{24}, porque $c \leq 24$. Es decir, el espacio real de semillas es $[0, 2^{24})$: unos 16,7 millones de posibilidades, frente a los 2^{256} que uno esperaría. Ese espacio se recorre por fuerza bruta en segundos.

La distribución no es uniforme. Como c se elige uniforme entre 16 y 24 y luego el valor es uniforme en $[0, 2^c)$, los números pequeños aparecen bajo más escalas y por tanto son más probables. Formalmente,

$$\Pr[\texttt{valor} = v] = \frac{1}{9}\sum_{c=16}^{24} \frac{\mathbf{1}[\,v < 2^c\,]}{2^c}, \qquad \Pr[\texttt{valor} < 2^{16}] = \frac{1}{9}\left(2 - 2^{-8}\right) \approx 0{,}22.$$

O sea, más de uno de cada cinco valores cae en el 0,4 % más bajo del rango.

Cuántos intentos esperamos. Si buscamos desde 0 hacia arriba, el número de intentos es $N = \texttt{valor} + 1$. Calculamos

$$\mathbb{E}[2^c] = \frac{2^{25} - 2^{16}}{9} \approx 3\,720\,988, \qquad \mathbb{E}[\texttt{valor}] \approx 1\,860\,494, \qquad \boxed{\mathbb{E}[N] \approx 1\,860\,495}.$$

En el peor caso tendríamos que llegar a $2^{24} \approx 1{,}68 \times 10^7$. Para encontrar la semilla solo necesitamos el archivo cifrado con $i = 0$: por cada candidato derivamos $K_0 = \mathrm{HMAC}(\text{candidato}, 0)$ e intentamos descifrar; el *tag* de GCM nos dice cuándo acertamos.

Lo que medimos. En una de nuestras corridas la semilla se encontró tras 150 072 intentos en 1,42 s, a unos $1{,}1 \times 10^5$ candidatos por segundo; en otra bastaron 13 619 intentos en 0,10 s. Las cifras varían porque dependen de la semilla concreta, pero siempre quedan muy por debajo del peor caso. A esa velocidad, incluso agotar todo el espacio tomaría alrededor de dos minutos y medio.

Romper la semilla frente a descifrar los archivos. Conviene separar los dos costos. Recuperar s_V cuesta del orden de $\mathbb{E}[N] \approx 1{,}9$ millones de operaciones; descifrar los M archivos, una vez que ya tenemos la semilla, cuesta solo M operaciones. El trabajo duro es, con diferencia, romper la semilla: para $M = 3$ archivos la proporción es de unas 620 000 a 1. En la práctica se paga una vez por quebrar la semilla y después descifrar cualquier cantidad de archivos es casi gratis. Toda la fortaleza de RSA y AES queda anulada por el eslabón débil, que es la entropía del generador.

6 El ataque del intermediario (MITM)

Para mostrar por qué hace falta autenticación, construimos un proxy que se sienta entre la víctima y el servidor y reenvía los mensajes. En el registro, sustituye la clave pública de la víctima pk_V por la suya, pk_M. El servidor, que no tiene forma de saber de quién es realmente esa clave, guarda pk_M creyendo que es de la víctima y cifra la semilla hacia ella. El proxy la descifra con su propia clave privada, se queda con s_V y luego la vuelve a cifrar con la pk_V auténtica para reenviársela a la víctima. La víctima completa su cifrado sin notar absolutamente nada, y el proxy ya tiene la semilla con la que descifrar todos los archivos.

Comprobamos este ataque de principio a fin: el proxy registra la semilla interceptada y descifra una copia de los archivos de la víctima, con el servidor luego apagado. Funciona precisamente porque RSA-OAEP garantiza confidencialidad hacia "quien tenga la clave", pero no dice nada sobre de quién es esa clave.

7 La defensa con firmas digitales

La defensa consiste en autenticar el origen de cada clave y de cada mensaje importante, usando firmas Ed25519 cuyas claves públicas de verificación están pre-compartidas. Tomamos cuatro decisiones de diseño.

Qué esquema de firma. Elegimos Ed25519 (sobre RSA-PSS) porque es determinista (no depende de aleatoriedad al firmar, lo que evita un tipo de fallo clásico), es rápido, produce firmas cortas de 64 bytes y su interfaz es difícil de usar mal. Firmar y cifrar quedan como funciones separadas: RSA-OAEP sigue protegiendo la semilla, y las firmas solo autentican.

Qué firmamos. Firmamos siempre el trío $id_V \parallel pk_{\text{cifrado}} \parallel$ rol. El id_V ata la firma a una víctima concreta; pk_{cifrado} es justo lo que viaja en claro en ese paso (la clave pública en el registro, o el criptograma de la semilla en el paso 2), de modo que si el proxy lo altera la firma deja de cuadrar; y el rol ("cliente" o "servidor") evita que una firma se reutilice en la otra dirección. Es la misma idea de los datos asociados del cifrado: atar cada dato a su contexto.

De dónde sale la confianza. Las claves públicas de verificación se reparten por un canal aparte y se guardan en `trusted_keys/`; no se envían por la red. Son el ancla de confianza: las firmas no crean confianza, solo la trasladan desde ese ancla. Por eso el servidor valida *únicamente* contra esas claves. Esto nos deja frenar los dos intentos del proxy: en el **Caso 1** cambia la clave pero no la firma, y la verificación falla; en el **Caso 2** forja una firma con su propia clave y adjunta su clave de verificación, pero como esa clave no está en `trusted_keys/`, el servidor la rechaza igual.

Cuándo se verifica. La comprobación ocurre *antes* de liberar la semilla. El servidor verifica la firma del cliente al recibir el registro y, si no cuadra, aborta sin generar ni enviar s_V. De forma simétrica, el cliente verifica la firma del servidor sobre el criptograma antes de descifrar la semilla. Comprobamos que, con el proxy en medio, el protocolo se detiene y la semilla nunca se roba; y que, con los participantes legítimos, el ciclo completo (cifrado, reinicio y recuperación por pago) funciona con normalidad.

8 Qué probamos

Para respaldar todo lo anterior dejamos un script, `ejecutar_pruebas.sh`, que ejecuta las cuatro pruebas pedidas y guarda cada salida en `evidencias/`. La primera prueba cifra los archivos, reinicia y recupera por pago: los tres archivos vuelven idénticos a los originales (comparación byte a byte y por SHA-256). La segunda es la investigación con el servidor apagado: encuentra la semilla por fuerza bruta y descifra todo. La tercera es el MITM sin defensa: el proxy intercepta la semilla y descifra el botín. La cuarta prueba la defensa: rechaza el Caso 1, rechaza el Caso 2 y, con los participantes legítimos, completa el ciclo con recuperación; en ninguno de los dos ataques el proxy logra quedarse con la semilla.

También dejamos todo listo para correr en contenedores: un `Dockerfile` ligero y un `docker-compose.yml` que aíslan servidor, proxy y cliente en una red interna, cada uno con su propio volumen de datos.

9 Conclusión

Nos quedamos con dos ideas. La primera es que un sistema puede usar criptografía impecable y aun así ser inseguro si un solo componente, aquí el generador de la semilla, tiene poca entropía; el cifrado no compensa ese fallo. La segunda es que cifrar no es autenticar: RSA-OAEP protege la semilla, pero sin firmas ancladas en una confianza previa un intermediario puede sustituir claves y quedarse con el secreto. Ambas ideas se ven de forma concreta en el código y en las evidencias que acompañan este informe.